

ANKARA ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ



BLM4538 IOS ile Mobil Uygulama Geliştirme II – Final Raporu

EnaReview: Ürün İnceleme – Puanlama Uygulaması

<https://github.com/oguzzaydogaan/enareview>

Oğuzhan Aydoğan

22290086

09.06.2026

1. PROJENİN AMACI VE KAPSAMI

Bu proje kapsamında; e-ticaret platformlarındaki güvenilirlik sorununu çözmek amacıyla, gerçek kullanıcı doğrulama sistemine sahip ve yapay zeka destekli bir ürün analiz platformu geliştirilmiştir. Projenin temel hedefi, SMS OTP ile kimlik doğrulaması yapılmış gerçek kullanıcıların ürün değerlendirmesi yapabildiği, bot hesapların engellendiği ve kullanıcıya binlerce yorumu okumak yerine AI özetleri sunan bir mobil uygulama ortaya koymaktır.

2. GERÇEKLEŞTİRİLEN SİSTEM MİMARİSİ

Sistem, yüksek ölçeklenebilirlik hedeflenerek asenkron ve dağıtık bir yapıda kurgulanmış ve 10 haftalık geliştirme süreci sonunda tüm planlanan bileşenler hayata geçirilmiştir.

- Mobil Uygulama: React Native + Expo ile geliştirildi. Kullanıcılar ürünleri listeleyebilir, beğenebilir, yorum yapabilir ve AI özetlerini görüntüleyebilir.
- API Katmanı: .NET 8 Web API kullanılarak inşa edildi. Servis katmanı, DTO pattern ve RESTful endpoint yapısıyla organize edildi.
- Mesaj Kuyruğu (RabbitMQ): Yorum ekleme işlemleri doğrudan veritabanına yazılmak yerine durable kuyruğa aktarıldı. Manual Acknowledgement ile veri kaybı sıfıra indirildi.
- Önbellek ve Sayaç (Redis): Popüler ürünlerin beğeni sayaçları Upstash Redis üzerinden atomik INCR/DECR komutlarıyla milisaniyeler içinde güncellendi. RedisSyncService, her 10 saniyede bir dirty kayıtları PostgreSQL'e yazdı.
- Yapay Zeka (Gemini): Google Gemini 2.5 Flash Lite modeli ile yorum özetleme entegre edildi. Özetler veritabanında önbelleklenerek her 5 yeni yorumda bir yenilenmektedir.

3. TEKNİK UYGULAMA VE TEKNOLOJİ SEÇİMLERİ

Bileşen	Seçilen Teknoloji	Uygulama Stratejisi
Kimlik Doğrulama	Firebase Auth	SMS OTP ile tek seferlik telefon doğrulaması yapıldı. Sonraki girişlerde kullanıcı adı/şifre + JWT kullanıldı.
Veritabanı	PostgreSQL	İlişkisel veriler (Kullanıcı, Ürün, Yorum, Like/Dislike, ProductSummary) Entity Framework Core 8 + Npgsql ile yönetildi.
Yapay Zeka	Gemini 2.5 Flash Lite	Google AI Studio ücretsiz kotası dahilinde HttpClient ile doğrudan REST API çağırısı yapıldı. Harici kütüphane kullanılmadı.
Asenkron İşlem	RabbitMQ	Yorum oluşturma işlemleri Producer/Consumer pattern ile asenkronize edildi. Durable queue + persistent mesaj yapısı kuruldu.
Performans	Upstash Redis	Write buffer pattern uygulandı: beğeni sayaçları Redis'te, kullanıcı-ürün ilişkisi PostgreSQL'de tutuldu.
Mobil Frontend	React Native + Expo	NativeWind (TailwindCSS) ile yeşil-beyaz tema, Axios ile API iletişimi, expo-secure-store ile güvenli token saklama.
Güvenlik	.env + BCrypt	Tüm API key ve connection string'ler .env dosyasından okundu. Kullanıcı şifreleri BCrypt.Net ile hashlendi.

4. UYGULANAN İŞ AKIŞLARI

4.1. SMS Tabanlı Kayıt Süreci

Kullanıcı kayıt aşamasında telefon numarası Firebase Auth üzerinden SMS OTP ile doğrulandı. Bu işlem yalnızca bir kez gerçekleştirildi; doğrulama tamamlandıktan sonra kullanıcı 'Verified User' statüsü kazandı. Sonraki girişlerde kullanıcı adı ve şifre ile giriş yapıldı, SMS maliyeti minimumda tutuldu.

4.2. Asenkron Yorum İşleme (RabbitMQ)

Yorum ekleme isteği geldiğinde backend 202 Accepted döndü ve yorum RabbitMQ kuyruğuna aktarıldı. ReviewConsumerService arka planda kuyruğu dinleyerek yorumu PostgreSQL'e yazdı. Başarılı yazma sonrasında Manual Acknowledgement ile mesaj kuyruktan silindi.

4.3. Yapay Zeka ile Otomatik Özetleme

Bir ürüne ilk yorum geldiğinde ReviewConsumerService otomatik olarak GeminiService'i çağırdı ve özet ProductSummary tablosuna kaydedildi. Sonraki her 5 yeni yorumda özet yenilendi. Bu lazy processing yaklaşımıyla gereksiz Gemini API çağrıları ve token tüketimi engellendi.

4.4. Redis ile Beğeni Yönetimi

Beğeni/beğenmeme işlemlerinde sayaçlar Redis'te atomik INCR/DECR ile güncellendi. Kullanıcı-ürün ilişkisi (kimin beğendiği) PostgreSQL'de tutuldu. RedisSyncService her 10 saniyede bir dirty set'teki ürünleri tespit edip PostgreSQL'e toplu güncelleme yaptı.

5. BÜTÇE VE KAYNAK YÖNETİMİ

Projenin tamamı sıfır maliyetle aşağıdaki ücretsiz servisler kullanılarak geliştirildi:

- **Firestore Spark Plan:** SMS OTP doğrulaması için aylık ücretsiz kota dahilinde kullanıldı.
- **Google AI Studio:** Günlük 1.500 istek ücretsiz kotası dahilinde Gemini 2.5 Flash Lite modeli kullanıldı.
- **Upstash Redis:** REST API tabanlı ücretsiz Redis katmanı ile beğeni sayaçları yönetildi.
- **Self-Hosting (Local):** PostgreSQL ve RabbitMQ geliştirme sürecinde yerel makinede çalıştırıldı.

6. KARŞILAŞILAN RİSKLER VE ÇÖZÜMLER

AI Model Uyumsuzlukları: gemma-3-27b-it modeli 404 döndürdü, gemma-4-31b-it ise chain-of-thought çıktısı nedeniyle kullanılamaz oldu. Çözüm: gemini-2.5-flash-lite modeli seçildi; temiz metin üretiyor ve ücretsiz kotaya sahip.

Veri Tutarlılığı (Redis ↔ PostgreSQL): Beğeni sayaçları Redis'te hızlı güncellenmesine karşın veritabanıyla senkronizasyon gerekiyordu. Çözüm: SETNX ile atomik ilk değer ataması ve dirty set takibi ile RedisSyncService periyodik toplu güncelleme yaptı.

Yüksek Frekanslı Yorum Yazma Yüğü: Doğrudan DB yazma yaklaşımı ölçeklenebilir değildi. Çözüm: RabbitMQ ile yazma yükü zamana yayıldı, kullanıcı anında yanıt aldı.

React State Güncelleme Gecikmesi (Filtre Çift Tıklama Hatası): Filtre chip'lerine çift tıklamada state güncel olmuyor, eski değer API'ye gönderiliyordu. Çözüm: Filtre değerleri ref üzerinde anlık tutuldu, state sadece render için kullanıldı.

7. HAFTALIK GELİŞTİRME SÜRECİ

7.1. 1. Hafta: Temel Yapılandırma ve Kayıt Süreçleri

- Arayüz: TailwindCSS kurulumu yapıldı, stil standartları oluşturuldu.
- Auth: FirebaseAuth entegrasyonu tamamlandı. Kayıt olma aşamasına SMS doğrulama (OTP) adımı eklendi.

7.2. 2. Hafta: Güvenli Oturum Yönetimi ve Navigasyon

- Oturum: JWT tabanlı giriş (login) ve otomatik token yenileme mekanizmaları kuruldu.
- Veri Güvenliği: Kullanıcı token'larının cihaz üzerinde güvenli saklanması için expo-secure-store entegre edildi.
- Navigasyon: Giriş sonrası 'Welcome' sayfası oluşturuldu. router.replace ile oturum sonrası geri dönüşler engellendi.

7.3. 3. Hafta: Veri Modelleri ve API Servis Yapılandırması

- Backend & Veri Mimarisi: Products, Reviews, Product Like ve Product Dislike entity yapıları oluşturuldu. Temel API endpoint'leri geliştirildi.
- Frontend Refactoring (Axios Geçişi): fetch yapısından Axios'a geçildi. BaseURL ve Authorization headers global konfigürasyonu ile her entity için ayrı servis katmanı tanımlandı.

7.4. 4. Hafta: Dinamik Veri Yönetimi ve Kullanıcı Arayüzü Geliştirme

- Backend (Performans İyileştirmesi): Ürün listeleme yapısı Pagination (sayfalama) yapısına geçirildi.
- Frontend: Products, Product Details ve Profile sayfaları geliştirildi. Yeni API endpoint'leri ile entegrasyon sağlandı.
- Test ve Veri: Postman Collection Runner ile Pre-request Script kullanılarak 100 adet test verisi otomatik olarak sisteme aktarıldı.

7.5. **5. Hafta: Ölçeklenebilir Mesaj Kuyruğu Yönetimi (RabbitMQ)**

- Backend (RabbitMQ Entegrasyonu): Yorum ekleme işlemleri doğrudan veritabanına yazılmak yerine asenkron yapıyla önce kuyruğa aktarıldı.
- Veri Güvenliği ve Dayanıklılık: Kuyruklar 'Durable', mesajlar 'Persistent' olarak yapılandırıldı.
- Background Service (Consumer): Kuyruğa alınan yorumları arka planda dinleyen ve Manual Acknowledgement mekanizmasıyla veritabanına işleyen Consumer servisi geliştirildi.

7.6. **6. Hafta: Görsel Kimlik Güncellemesi ve Veri Modeli Genişletme**

- Backend: Ürün yapısı resim (Image) ve kategori (Category) alanlarıyla genişletildi. İlgili endpoint'ler güncellendi.
- Frontend: Uygulama görsel kimliği kırmızı-beyaz temasından yeşil-beyaz temasına geçirildi. Kategori ve resim verileri arayüze entegre edildi.

7.7. **7. Hafta: Redis ile Hız ve Senkronizasyon**

- Backend (Redis Entegrasyonu — Upstash): Beğeni/beğenmeme sayaçları atomik INCR/DECR komutlarıyla Redis üzerinde güncellenir hale getirildi. Kullanıcı-ürün ilişkisi PostgreSQL'de korundu.
- Senkronizasyon Servisi: Her 10 saniyede bir dirty ürünleri PostgreSQL'e toplu yazan RedisSyncService geliştirildi.
- Güvenlik: API anahtarları ve bağlantı bilgileri appsettings.json'dan kaldırılarak .env dosyasına taşındı.

7.8. **8. Hafta: AI Entegrasyonu**

- Backend (AI Servis Altyapısı): IAIService arayüzü ve GeminiService implementasyonu oluşturuldu. HttpClient ile doğrudan Gemini REST API'sine bağlandı; timeout yönetimi servis katmanında tutuldu.

- Akıllı Özetleme: Yorum sayısı eşiği geçen ürünlerde ürün detay sayfası yüklenirken yorumlar Gemini 2.5 Flash Lite modeline gönderildi ve 2-3 cümlelik özet üretildi.
- Frontend: Ürün detay sayfasına yeşil arka planlı 'AI Özeti' kartı eklendi.

7.9. **9. Hafta: Arama, Filtreleme, Sıralama ve AI Özet Ön bellekleme**

- Backend (AI Özet Ön bellekleme): ProductSummary entity'si ve migration oluşturuldu. Özetleme mantığı ReviewConsumerService'e taşındı: ilk yorumda özet üretilir, her 5 yeni yorumda yenilenir.
- Backend (Ürün Arama ve Filtreleme): search (içerik araması), sortBy (newest / mostReviewed / mostLiked) ve categoryId query parametreleri eklendi. EF Core LINQ üzerinde veritabanı katmanında çalışmakta; pagination ile tam uyumlu.
- Frontend: Arama çubuğu (400ms debounce), sıralama chip'leri ve kategori filtreleri eklendi. Filtre değerlerinin ref üzerinde tutulmasıyla çift tıklama hatası giderildi; arama çubuğu FlatList dışına alınarak klavye kapanma sorunu çözüldü.

7.10. **10. Hafta: Genel Uygulama Tanıtımı ve Test**

- Uçtan Uca (End-to-End) Test: Yeni kullanıcı kaydı, giriş ve JWT yönetimi, ürün listeleme ve infinite scroll, arama/sıralama/filtreleme, AI özeti görüntüleme, beğeni toggle (Redis + DB senkronizasyonu) ve yorum ekleme/silme akışları gerçek cihaz üzerinde test edildi.
- Performans ve Dayanıklılık Doğrulaması: Redis write buffer ve RedisSyncService'in doğru çalıştığı, RabbitMQ kuyruğunun durable yapısının veri kaybını önlediği teyit edildi.
- Final Tanıtımı: 9 haftalık geliştirme sürecinde oluşturulan tüm özellikler bir arada çalışır hâlde son kez gözden geçirildi.

8. EKRAN GÖRÜNTÜLERİ

